

codex-windows / 019f2174-3ad3-7700-9f93-3841c0733077

Metadata

```
- Source: `codex-windows`  
- Kind: `codex`  
- Source file: `C:\Users\avidu\.codex\archived_sessions\rollout-2026-07-02T11-41-33-019f2174-3ad3-7700-9f93-3841c0733077.jsonl`  
- SHA-256: `560660099f46836bfcc91e2bb32c7708343f13e7ea5297991aeb5f753385acc5`  
- Source modified: `2026-07-02T06:11:33+00:00`  
- Imported at: `2026-07-05T16:48:16+00:00`  
- cli_version: `0.142.5`  
- cwd: `C:\Users\avidu\Projects\badminton-highlight-indexer`  
- model_provider: `openai`  
- session_id: `019f2174-3ad3-7700-9f93-3841c0733077`  
- source: `vscode`
```

Transcript

1. user (2026-07-02T06:11:33.205Z)

Read C:\Users\avidu\OneDrive\Documents\Claude-Sync-Setup-Guide.md. This desktop is the canonical source. Run Step 1 (seed) then Step 2 (link) from that guide in PowerShell ? copy my ~/.claude CLAUDE.md and project memories into OneDrive\Documents\claude-sync and link this machine to them. You may not have admin rights; if the CLAUDE.md symlink fails, the @import stub fallback is fine. Show me the output and confirm what got seeded and linked.

2. assistant (2026-07-02T06:11:33.205Z)

I'll start by reading the setup guide to understand exactly what Step 1 and Step 2 involve before running anything.

3. assistant (2026-07-02T06:11:33.205Z)

```
[external_agent_tool_call: Read]  
file: C:\Users\avidu\OneDrive\Documents\Claude-Sync-Setup-Guide.md  
[/external_agent_tool_call]
```

4. assistant (2026-07-02T06:11:33.205Z)

```
[external_agent_tool_result]  
1      # Sharing Claude Code memory + global instructions across machines  
2  
3      **Goal:** make your global instructions (`CLAUDE.md`) and per-project memories  
4      follow you between your **desktop** and **laptop**, via OneDrive.  
5  
6      ## Why this is needed  
7  
8      Claude Code stores everything on the local disk under `~/.claude` (i.e.  
9      `C:\Users\avidu\.claude\`). Your Anthropic account only handles login/billing ?  
10     it does **not** sync these files. So each machine builds its own memory from  
11     scratch. The fix: keep one canonical copy in OneDrive and point both machines  
12     at it with links.  
13  
14     ## What syncs (and what must NOT)  
15  
16     | Synced via OneDrive | Stays per-machine (never sync) |  
17     |---|---|  
18     | `CLAUDE.md` (your "disciplined states", e.g. the pre-LGTM gate) | `~/.claude.json`  
(auth token + MCP config) |  
19     | `~/.claude/projects/*/memory/` (memory facts) | `sessions/`, `shell-snapshots/`,  
`session-env/`, `ide/`, `backups/` |  
20
```

```

21  **Sync folder (same logical path on both machines):**
22  `C:\Users\avidu\OneDrive\Documents\claudio-sync\`
23  (Both machines have Documents redirected into OneDrive, so this resolves to the
24  same synced folder on each.)
25
26  > The desktop is the **canonical source** ? seed from it first, then link the
27  > laptop. Each link step backs up whatever it replaces as `*.pre-sync.bak`.
28
29  ---
30
31  ## Step 1 ? On the DESKTOP: copy your state into OneDrive
32
33  Plain (non-admin) Windows PowerShell 5:
34
35  ```powershell
36  $claudio = Join-Path $env:USERPROFILE '.claudio'
37  $sync    = Join-Path ([Environment]::GetFolderPath('MyDocuments')) 'claudio-sync'
38  New-Item -ItemType Directory -Force (Join-Path $sync 'memory') | Out-Null
39
40  # Global instructions
41  Copy-Item (Join-Path $claudio 'CLAUDE.md') (Join-Path $sync 'CLAUDE.md') -Force
42
43  # Every project's memory\ folder, named by project
44  Get-ChildItem (Join-Path $claudio 'projects') -Directory | ForEach-Object {
45      $mem = Join-Path $_.FullName 'memory'
46      if (Test-Path $mem) {
47          $name = ($_.Name -split 'Projects-')[-1]
48          $dest = Join-Path $sync ('memory\' + $name)
49          New-Item -ItemType Directory -Force $dest | Out-Null
50          Copy-Item (Join-Path $mem '*') $dest -Recurse -Force
51          Write-Host "copied memory: $name"
52      }
53  }
54  Write-Host "Seeded -> $sync"
55  ```
56
57  ## Step 2 ? On the DESKTOP: point it at the synced copies
58
59  **Open PowerShell as Administrator** (so the `CLAUDE.md` symlink succeeds
60  cleanly; junctions don't need admin, but elevating the whole block is simplest):
61
62  ```powershell
63  $claudio = Join-Path $env:USERPROFILE '.claudio'
64  $sync    = Join-Path ([Environment]::GetFolderPath('MyDocuments')) 'claudio-sync'
65
66  # CLAUDE.md -> symlink into OneDrive (falls back to an @import stub if not elevated)
67  $src = Join-Path $sync 'CLAUDE.md'
68  $dst = Join-Path $claudio 'CLAUDE.md'
69  if (Test-Path $src) {
70      if (Test-Path $dst) { Copy-Item $dst "$dst.pre-sync.bak" -Force; Remove-Item $dst
-Force }
71      try {
72          New-Item -ItemType SymbolicLink -Path $dst -Target $src -ErrorAction Stop | Out-Null
73          Write-Host "CLAUDE.md -> symlink"
74      } catch {
75          [IO.File]::WriteAllText($dst, "@$src"); Write-Host "CLAUDE.md -> import stub (no
admin)"
76      }
77  } else { Write-Host "No CLAUDE.md in sync folder - run Step 1 first" }
78
79  # memory -> directory junctions (no admin needed)
80  Get-ChildItem (Join-Path $claudio 'projects') -Directory | ForEach-Object {
81      $name = ($_.Name -split 'Projects-')[-1]
82      $srcM = Join-Path $sync ('memory\' + $name)
83      $dstM = Join-Path $_.FullName 'memory'

```

```

84         if (Test-Path $srcM) {
85             if (Test-Path $dstM) { Copy-Item $dstM "$dstM.pre-sync.bak" -Recurse -Force; Remove-
Item $dstM -Recurse -Force }
86             New-Item -ItemType Junction -Path $dstM -Target $srcM | Out-Null
87             Write-Host "memory -> junction: $name"
88         }
89     }
90     ...
91
92     ## Step 3 ? On the LAPTOP (after OneDrive has synced the folder down)
93
94     Wait until `C:\Users\av...
[/external_agent_tool_result]

```

5. assistant (2026-07-02T06:11:33.205Z)

I've read the guide. Before running anything, let me capture a clear "before" snapshot ? this matters because Step 2 is destructive (it replaces your real `CLAUDE.md` and each `memory\` folder with links, backing them up first), and I want to confirm path resolution and current link state before I touch anything.

6. assistant (2026-07-02T06:11:33.205Z)

```

[external_agent_tool_call: PowerShell]
description: Inspect current ~/.claude state and resolve sync paths (read-only)
command: $claude = Join-Path $env:USERPROFILE '.claude'
$myDocs = [Environment]::GetFolderPath('MyDocuments')
$sync = Join-Path $myDocs 'claude-sync'

Write-Host "=== Path resolution ==="
Write-Host "USERPROFILE : $env:USERPROFILE"
Write-Host "claude dir : $claude"
Write-Host "MyDocuments : $myDocs"
Write-Host "sync target : $sync"
Write-Host ""

Write-Host "=== ~/.claude CLAUDE.md current state ==="
$dst = Join-Path $claude 'CLAUDE.md'
if (Test-Path $dst) {
    $item = Get-Item $dst -Force
    $isLink = [bool]($item.Attributes -band [IO.FileAttributes]::ReparsePoint)
    Write-Host ("CLAUDE.md exists. ReparsePoint(link)={0} LinkTarget={1} Length={2} bytes" -f
$isLink, $item.LinkTarget, $item.Length)
} else { Write-Host "CLAUDE.md does NOT exist" }
Write-Host ""

Write-Host "=== Projects with memory\ folders ==="
Get-ChildItem (Join-Path $claude 'projects') -Directory | ForEach-Object {
    $mem = Join-Path $_.FullName 'memory'
    if (Test-Path $mem) {
        $mi = Get-Item $mem -Force
        $isJ = [bool]($mi.Attributes -band [IO.FileAttributes]::ReparsePoint)
        $name = ($_.Name -split 'Projects-')[1]
        $cnt = (Get-ChildItem $mem -Recurse -File -ErrorAction SilentlyContinue | Measure-
Object).Count
        Write-Host (" key='{0}' junction={1} files={2} rawdir='{3}'" -f $name, $isJ, $cnt,
$.Name)
    }
}
Write-Host ""

Write-Host "=== claude-sync folder pre-existing? ==="
if (Test-Path $sync) {
    Write-Host "EXISTS: $sync"
    Get-ChildItem $sync -Force | Select-Object Mode, Length, Name | Format-Table -AutoSize | Out-

```

```
String | Write-Host
} else { Write-Host "Does not exist yet (will be created in Step 1)" }
[/external_agent_tool_call]
```

7. assistant (2026-07-02T06:11:33.206Z)

```
[external_agent_tool_result]
=== Path resolution ===
USERPROFILE : C:\Users\avidu
claude dir   : C:\Users\avidu\.claude
MyDocuments : C:\Users\avidu\OneDrive\Documents
sync target  : C:\Users\avidu\OneDrive\Documents\claude-sync

=== ~/.claude CLAUDE.md current state ===
CLAUDE.md exists. ReparsePoint(link)=False LinkTarget= Length=4091 bytes

=== Projects with memory\ folders ===
key='-NET' junction=False files=2 rawdir='c--Users-avidu-Projects--NET'
key='badminton-highlight-indexer' junction=False files=25 rawdir='C--Users-avidu-Projects-badminton-highlight-indexer'
key='muneem' junction=False files=12 rawdir='C--Users-avidu-Projects-muneem'
key='sports-data-collector' junction=False files=3 rawdir='c--Users-avidu-Projects-sports-data-collector'

=== claude-sync folder pre-existing? ===
Does not exist yet (will be created in Step 1)
[/external_agent_tool_result]
```

8. assistant (2026-07-02T06:11:33.206Z)

Clean baseline confirmed:

- `MyDocuments` correctly resolves into OneDrive (`C:\Users\avidu\OneDrive\Documents`), so the sync folder lands in the synced location.
- `CLAUDE.md` is a real 4091-byte file (not yet a link).
- 4 projects have memory folders (`-NET`, `badminton-highlight-indexer`, `muneem`, `sports-data-collector`) ? all real dirs, no junctions yet, no key collisions.
- `claude-sync` doesn't exist yet.

Now running ****Step 1 (seed)**** verbatim from the guide ? this only copies **out** to OneDrive, so it's non-destructive to `~/.claude`.

9. assistant (2026-07-02T06:11:33.206Z)

```
[external_agent_tool_call: PowerShell]
description: Step 1 ? seed CLAUDE.md and project memories into OneDrive claude-sync
command: $claude = Join-Path $env:USERPROFILE '.claude'
$sync    = Join-Path ([Environment]::GetFolderPath('MyDocuments')) 'claude-sync'
New-Item -ItemType Directory -Force (Join-Path $sync 'memory') | Out-Null
```

Global instructions

```
Copy-Item (Join-Path $claude 'CLAUDE.md') (Join-Path $sync 'CLAUDE.md') -Force
```

Every project's memory\ folder, named by project

```
Get-ChildItem (Join-Path $claude 'projects') -Directory | ForEach-Object {
    $mem = Join-Path $_.FullName 'memory'
    if (Test-Path $mem) {
        $name = ($_.Name -split 'Projects-')[-1]
        $dest = Join-Path $sync ('memory\' + $name)
        New-Item -ItemType Directory -Force $dest | Out-Null
        Copy-Item (Join-Path $mem '*') $dest -Recurse -Force
        Write-Host "copied memory: $name"
    }
}
```

```
Write-Host "Seeded -> $sync"
[/external_agent_tool_call]
```

10. assistant (2026-07-02T06:11:33.206Z)

```
[external_agent_tool_result]
copied memory: -NET
copied memory: badminton-highlight-indexer
copied memory: muneem
copied memory: sports-data-collector
Seeded -> C:\Users\avidu\OneDrive\Documents\claude-sync
[/external_agent_tool_result]
```

11. assistant (2026-07-02T06:11:33.206Z)

Step 1 reported success for all four projects. Let me verify the seed actually landed correctly ? comparing file counts source vs. seeded, and confirming `CLAUDE.md` copied byte-for-byte.

12. assistant (2026-07-02T06:11:33.206Z)

```
[external_agent_tool_call: PowerShell]
description: Verify seed: hash CLAUDE.md and compare memory file counts
command: $claude = Join-Path $env:USERPROFILE '.claude'
$sync = Join-Path ([Environment]::GetFolderPath('MyDocuments')) 'claude-sync'

Write-Host "=== CLAUDE.md compare ==="
$srcMd = Join-Path $claude 'CLAUDE.md'
$dstMd = Join-Path $sync 'CLAUDE.md'
$sh = (Get-FileHash $srcMd -Algorithm MD5).Hash
$dh = (Get-FileHash $dstMd -Algorithm MD5).Hash
Write-Host ("source {0} bytes md5={1}" -f (Get-Item $srcMd).Length, $sh)
Write-Host ("synced {0} bytes md5={1}" -f (Get-Item $dstMd).Length, $dh)
Write-Host ("MATCH = {0}" -f ($sh -eq $dh))
Write-Host ""

Write-Host "=== Per-project memory file-count compare (source vs synced) ==="
Get-ChildItem (Join-Path $claude 'projects') -Directory | ForEach-Object {
    $mem = Join-Path $_.FullName 'memory'
    if (Test-Path $mem) {
        $name = ($_.Name -split 'Projects-')[-1]
        $dest = Join-Path $sync ('memory\' + $name)
        $sc = (Get-ChildItem $mem -Recurse -File -ErrorAction SilentlyContinue | Measure-Object).Count
        $dc = (Get-ChildItem $dest -Recurse -File -ErrorAction SilentlyContinue | Measure-Object).Count
        $ok = if ($sc -eq $dc) { 'OK' } else { 'MISMATCH' }
        Write-Host (" {0,-30} source={1,-3} synced={2,-3} {3}" -f $name, $sc, $dc, $ok)
    }
}
Write-Host ""
Write-Host "=== Seeded folder tree (top 2 levels) ==="
Get-ChildItem $sync -Force | Select-Object Mode,Name | Format-Table -AutoSize | Out-String |
Write-Host
Get-ChildItem (Join-Path $sync 'memory') -Force | Select-Object Mode,Name | Format-Table
-AutoSize | Out-String | Write-Host
[/external_agent_tool_call]
```

13. assistant (2026-07-02T06:11:33.206Z)

```
[external_agent_tool_result]
=== CLAUDE.md compare ===
source 4091 bytes md5=EA066A6CA0ADF9B5A8631EC7D386AE3A
synced 4091 bytes md5=EA066A6CA0ADF9B5A8631EC7D386AE3A
MATCH = True
```

=== Per-project memory file-count compare (source vs synced) ===

-NET	source=2	syncd=2	OK
badminton-highlight-indexer	source=25	syncd=25	OK
muneem	source=12	syncd=12	OK
sports-data-collector	source=3	syncd=3	OK

=== Seeded folder tree (top 2 levels) ===

```
Mode Name
----
lar-- memory
la--- CLAUDE.md
```

```
Mode Name
----
lar-- -NET
lar-- badminton-highlight-indexer
lar-- muneem
lar-- sports-data-collector
[/external_agent_tool_result]
```

14. assistant (2026-07-02T06:11:33.206Z)

Seed verified ? CLAUDE.md is byte-identical (same MD5) and every project's memory file count matches exactly (2/25/12/3).

Now **Step 2 (link)** ? the destructive step. It backs up the real `CLAUDE.md` ? `.pre-sync.bak` and each `memory\` ? `memory.pre-sync.bak` **before** replacing them with links, and falls back to an `@import` stub if the symlink needs admin I don't have. Running it verbatim:

15. assistant (2026-07-02T06:11:33.206Z)

```
[external_agent_tool_call: PowerShell]
description: Step 2 ? link ~/.claude CLAUDE.md and memories to OneDrive copies
command: $claude = Join-Path $env:USERPROFILE '.claude'
$sync = Join-Path ([Environment]::GetFolderPath('MyDocuments')) 'claude-sync'
```

CLAUDE.md -> symlink into OneDrive (falls back to an @import stub if not elevated)

```
$src = Join-Path $sync 'CLAUDE.md'
$dst = Join-Path $claude 'CLAUDE.md'
if (Test-Path $src) {
    if (Test-Path $dst) { Copy-Item $dst "$dst.pre-sync.bak" -Force; Remove-Item $dst -Force }
    try {
        New-Item -ItemType SymbolicLink -Path $dst -Target $src -ErrorAction Stop | Out-Null
        Write-Host "CLAUDE.md -> symlink"
    } catch {
        [IO.File]::WriteAllText($dst, "@$src"); Write-Host "CLAUDE.md -> import stub (no admin)"
    }
} else { Write-Host "No CLAUDE.md in sync folder - run Step 1 first" }
```

memory -> directory junctions (no admin needed)

```
Get-ChildItem (Join-Path $claude 'projects') -Directory | ForEach-Object {
    $name = ($_.Name -split 'Projects-')[1]
    $srcM = Join-Path $sync ('memory\' + $name)
    $dstM = Join-Path $_.FullName 'memory'
    if (Test-Path $srcM) {
        if (Test-Path $dstM) { Copy-Item $dstM "$dstM.pre-sync.bak" -Recurse -Force; Remove-Item $dstM -Recurse -Force }
        New-Item -ItemType Junction -Path $dstM -Target $srcM | Out-Null
        Write-Host "memory -> junction: $name"
    }
}
```

```
}  
[/external_agent_tool_call]
```

16. assistant (2026-07-02T06:11:33.206Z)

```
[external_agent_tool_result]  
CLAUDE.md -> symlink  
memory -> junction: -NET  
memory -> junction: badminton-highlight-indexer  
memory -> junction: muneem  
memory -> junction: sports-data-collector  
[/external_agent_tool_result]
```

17. assistant (2026-07-02T06:11:33.206Z)

Step 2 succeeded cleanly ? the `CLAUDE.md` **symlink worked** (Developer Mode/admin is available, so no `@import` stub was needed), and all four memory folders are now junctions. Let me verify the links resolve correctly, content reads through them, and the backups are in place.

18. assistant (2026-07-02T06:11:33.206Z)

```
[external_agent_tool_call: PowerShell]  
description: Verify CLAUDE.md symlink and memory junctions resolve, plus backups exist  
command: $claude = Join-Path $env:USERPROFILE '.claude'  
$sync = Join-Path ([Environment]::GetFolderPath('MyDocuments')) 'claude-sync'  
  
Write-Host "=== CLAUDE.md link ==="  
$dst = Join-Path $claude 'CLAUDE.md'  
$i = Get-Item $dst -Force  
$isLink = [bool]($i.Attributes -band [IO.FileAttributes]::ReparsePoint)  
Write-Host ("ReparsePoint={0} LinkTarget={1}" -f $isLink, $i.LinkTarget)  
$readHash = (Get-FileHash $dst -Algorithm MD5).Hash  
$syncHash = (Get-FileHash (Join-Path $sync 'CLAUDE.md') -Algorithm MD5).Hash  
Write-Host ("read-through-link md5={0} sync md5={1} MATCH={2}" -f $readHash, $syncHash,  
($readHash -eq $syncHash))  
$bak = "$dst.pre-sync.bak"  
Write-Host ("backup exists: {0} ({1} bytes)" -f (Test-Path $bak), (if (Test-Path $bak) { (Get-  
Item $bak).Length } else { 'n/a' })))  
Write-Host ""  
  
Write-Host "=== memory junctions + backups ==="  
Get-ChildItem (Join-Path $claude 'projects') -Directory | ForEach-Object {  
    $name = ($_.Name -split 'Projects-')[-1]  
    $dstM = Join-Path $_.FullName 'memory'  
    if (Test-Path $dstM) {  
        $mi = Get-Item $dstM -Force  
        $isJ = [bool]($mi.Attributes -band [IO.FileAttributes]::ReparsePoint)  
        $tgt = $mi.Target  
        $cnt = (Get-ChildItem $dstM -Recurse -File -ErrorAction SilentlyContinue | Measure-  
Object).Count  
        $bak = "$dstM.pre-sync.bak"  
        $bakOk = Test-Path $bak  
        Write-Host (" {0,-30} junction={1} files-via-link={2} backup={3}" -f $name, $isJ, $cnt,  
$bakOk)  
        Write-Host ("          -> target: {0}" -f $tgt)  
    }  
}  
}  
[/external_agent_tool_call]
```

19. assistant (2026-07-02T06:11:33.206Z)

```
[external_agent_tool_result]  
=== CLAUDE.md link ===  
ReparsePoint=True LinkTarget=C:\Users\avidu\OneDrive\Documents\claude-sync\CLAUDE.md
```

```

read-through-link md5=EA066A6CA0ADF9B5A8631EC7D386AE3A sync
md5=EA066A6CA0ADF9B5A8631EC7D386AE3A MATCH=True
if:
Line |
13 | ? st ("backup exists: {0} ({1} bytes)" -f (Test-Path $bak),
(if (Test-Path ?
|
~~
| The term 'if' is
not recognized as a name of a cmdlet, function, script file, or executable program.
Check the spelling of the
name, or if a path was included, verify that the path is correct and try again.

=== memory junctions + backups ===
- NET junction=True files-via-link=2 backup=True
-> target: C:\Users\avidu\OneDrive\Documents\claude-sync\memory\ -NET
badminton-highlight-indexer junction=True files-via-link=25 backup=True
-> target: C:\Users\avidu\OneDrive\Documents\claude-sync\memory\badminton-highlight-
indexer
muneem junction=True files-via-link=12 backup=True
-> target: C:\Users\avidu\OneDrive\Documents\claude-sync\memory\muneem
sports-data-collector junction=True files-via-link=3 backup=True
-> target: C:\Users\avidu\OneDrive\Documents\claude-sync\memory\sports-data-collector
[/external_agent_tool_result]

```

20. assistant (2026-07-02T06:11:33.207Z)

All links resolve correctly. The one red error is just a PowerShell syntax slip in my `*verification*` line (you can't use ``if`` as an inline expression inside ``-f``) ? it didn't affect the setup, only skipped printing the CLAUDE.md backup size. Let me confirm that backup cleanly and list all the ``.pre-sync.bak`` safety copies:

21. assistant (2026-07-02T06:11:33.207Z)

```

[external_agent_tool_call: PowerShell]
description: Confirm CLAUDE.md backup and list all pre-sync backups
command: $claude = Join-Path $env:USERPROFILE '.claude'
$mdBak = Join-Path $claude 'CLAUDE.md.pre-sync.bak'
if (Test-Path $mdBak) {
    $b = Get-Item $mdBak
    Write-Host ("CLAUDE.md.pre-sync.bak EXISTS - {0} bytes md5={1}" -f $b.Length, (Get-FileHash
$mdBak -Algorithm MD5).Hash)
} else { Write-Host "CLAUDE.md.pre-sync.bak MISSING" }
Write-Host ""
Write-Host "=== All .pre-sync.bak backups under ~/.claude ==="
Get-ChildItem $claude -Recurse -Force -Filter '*.pre-sync.bak' -ErrorAction SilentlyContinue |
    ForEach-Object { Write-Host (" {0}" -f $_.FullName.Replace($claude, '~\.claude')) }
[/external_agent_tool_call]

```

22. assistant (2026-07-02T06:11:33.207Z)

```

[external_agent_tool_result]
CLAUDE.md.pre-sync.bak EXISTS - 4091 bytes md5=EA066A6CA0ADF9B5A8631EC7D386AE3A

=== All .pre-sync.bak backups under ~/.claude ===
~\.claude\CLAUDE.md.pre-sync.bak
~\.claude\projects\c--Users-avidu-Projects--NET\memory.pre-sync.bak
~\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory.pre-sync.bak
~\.claude\projects\C--Users-avidu-Projects-muneem\memory.pre-sync.bak
~\.claude\projects\c--Users-avidu-Projects-sports-data-collector\memory.pre-sync.bak
[/external_agent_tool_result]

```

23. assistant (2026-07-02T06:11:33.207Z)

Done ? both steps ran on this desktop (the canonical source) and I verified every link and backup. Here's exactly what happened:

Step 1 ? Seed (copy into OneDrive) ?

Created `C:\Users\avidu\OneDrive\Documents\claude-sync\` and copied your state in. Verified, not just reported:

```
| Seeded | Result |
|---|---|
| `CLAUDE.md` | 4091 bytes, **MD5 matches source** |
| `memory\NET` | 2/2 files ? |
| `memory\badminton-highlight-indexer` | 25/25 files ? |
| `memory\munem` | 12/12 files ? |
| `memory\sports-data-collector` | 3/3 files ? |
```

Step 2 ? Link (point `~/claude` at the synced copies) ?

- **CLAUDE.md** ? real symlink ? `...\claude-sync\CLAUDE.md`. The symlink succeeded (this machine has admin/Developer Mode), so **no** `@import` stub fallback was needed. Reading through the link returns the correct content (MD5 matches).
- All 4 `memory\` folders ? directory junctions pointing into `...\claude-sync\memory\<project>`, each reading back the right file counts (2/25/12/3) through the link.

Safety backups ? all originals preserved ?

Every replaced item was backed up first:

- `~/claude\CLAUDE.md.pre-sync.bak` (4091 bytes, MD5 matches the original)
- `~/claude\projects\NET\memory.pre-sync.bak`
- `~/claude\projects\badminton-highlight-indexer\memory.pre-sync.bak`
- `~/claude\projects\munem\memory.pre-sync.bak`
- `~/claude\projects\sports-data-collector\memory.pre-sync.bak`

To undo any of these: delete the link and rename the matching `*.pre-sync.bak` back (per the guide's "To undo" note).

What's next (not run ? these are out of scope for "desktop seed + link")

- **OneDrive upload:** the seeded files are written into your OneDrive folder but still need to **upload to the cloud** before the laptop can see them. Confirm via the OneDrive tray icon that `claude-sync\` shows the green "synced" check.
- **Laptop (Step 3):** once `claude-sync\` (with `CLAUDE.md` + `memory\<project>`) has synced **down** on the laptop, run the **same Step 2 block** there (elevated). It only links ? it won't overwrite this desktop's canonical content, and it backs up the laptop's current memory as `*.pre-sync.bak` first.

One thing to be aware of going forward: from now on, editing project memory on this machine writes **straight into OneDrive** (that's the point), and `~/claude/CLAUDE.md` will show up as a reparse point/symlink ? that's expected, not corruption.

24. user (2026-07-02T06:11:33.207Z)

that is expected behaviour! Thnks for maaking the che chaange

25. assistant (2026-07-02T06:11:33.207Z)

You're welcome! Glad it's working as expected. ?

Quick recap of where things stand: this desktop is now the canonical source ? `CLAUDE.md` symlinked and all 4 project memories junctioned into `OneDrive\Documents\claude-sync\`, with `*.pre-sync.bak` safety copies of everything. Whenever you're ready on the laptop, just wait for OneDrive to sync the folder down, then run the same Step 2 block there (elevated).

Happy to help with the laptop side when you get to it.